

UNIVERSITY OF GHANA
Department of Computer Science
MSc Computer Science

Feature-based Image Matching and Automatic Panorama Construction

Project report

Shadrack Bentil
sbentil005@st.ug.edu.gh
GitHub: @qbentil

August 2026

A classical OpenCV pipeline (detect, describe, match, RANSAC, warp, stitch) with an interactive walkthrough. The full two-column technical paper lives on the live app's Project report tab and is not this document.

1. Where to find the work

- Live application: <https://feature-based-panorama.streamlit.app/>
- Source (public): <https://github.com/qbentil/feature-based-panorama>
- Full technical paper (CVPR LaTeX): `report/feature_based_panorama.pdf` in the repository, also embedded on the app’s Project report tab.

This file is the short project packet: method, source excerpts, experiment figures, and screenshots of the running app. It is not the CVPR paper shown in the application.

The repository contains the pipeline under `src/`, the Streamlit walkthrough under `app_pages/`, Wikimedia Commons campus photographs (Balme Library, Dance Department, Night Market, Commonwealth Hall) with licences in `data/ATTRIBUTION.md`, and experiment tables in `report/results/`. The implementation does not call `cv2.Stitcher`, YOLO, or any pretrained recogniser.

2. Introduction

A panorama is a single wide image assembled from overlapping photographs of the same scene. Consumer cameras hide the geometry: they find the same landmarks in neighbouring frames, discard inconsistent correspondences, and warp one frame onto another. Those steps are standard computer vision—feature detection, description, matching, robust homography estimation, and blending—rather than a learned recogniser.

This project implements that chain explicitly so every decision is inspectable:

1. Detect distinctive keypoints and compute descriptors with SIFT and with ORB.
2. Match descriptors between overlapping pairs and visualise correspondences before and after RANSAC.
3. Estimate a homography, warp images into one coordinate system, and stitch a panorama from at least three views.
4. Compare the detectors on match count, inlier ratio, reprojection error, overlap SSIM, and time.
5. Measure how rotation, scale, illumination, and viewpoint change affect the inlier ratio.

The shared geometry lives in `src/`. Both the Streamlit app and `scripts/run_experiments.py` call the same `match_pair` and `stitch_images` entry points; the UI never reimplements the homography.

The pipeline is:

overlapping photos → prepare → detect and describe → match → RANSAC → homography → warp to the middle photo → feather blend → panorama.

3. Implementation

Each image is resized so the long edge is 1200 px, lightly Gaussian-smoothed, and optionally CLAHE-equalised on the grayscale copy used for detection. Colour is kept for warping and blending. Keypoints are capped at 2000 per image so SIFT and ORB are compared under the same budget.

3.1. Detect and describe (src/features.py)

SIFT builds a gradient-histogram descriptor designed to be stable under scale and rotation. ORB combines FAST corners with a rotated BRIEF binary descriptor; matching then uses Hamming distance.

```
def create_detector(name, n_features=2000):
    key = name.strip().upper()
    if key == "SIFT":
        return cv2.SIFT_create(nfeatures=n_features)
    if key == "ORB":
        return cv2.ORB_create(nfeatures=n_features)
```

Listing 1: Detector factory.

3.2. Match (src/matching.py)

Brute-force k NN ($k=2$) followed by Lowe's ratio test at 0.75. SIFT uses L_2 ; ORB uses Hamming.

```
knn = matcher.knnMatch(descriptors_a, descriptors_b, k=2)
good = []
for pair in knn:
    if len(pair) < 2:
        continue
    best, second = pair
    if best.distance < ratio * second.distance:
        good.append(best)
```

Listing 2: Lowe ratio test on k NN pairs.

3.3. RANSAC homography (src/homography.py)

`cv2.findHomography` with a 5 px reprojection threshold. Consecutive pair maps are composed into the middle image's frame so the canvas does not drift to one end of the sequence.

```
H, mask = cv2.findHomography(
    pts_src.reshape(-1, 1, 2),
    pts_dst.reshape(-1, 1, 2),
    method=cv2.RANSAC,
    ransacReprojThreshold=ransac_thresh,
)
```

Listing 3: Robust homography.

3.4. Warp and stitch (src/stitch.py)

Each view is `warpPerspective`-mapped onto a canvas large enough for all warped corners. Overlap is feather-blended with a distance-transform weight so seams do not appear as hard cuts.

```
dist = cv2.distanceTransform(binary, cv2.DIST_L2, 5)
w = dist.astype(np.float32)
acc += img.astype(np.float32) * w[..., None]
weight += w
blended = acc / np.maximum(weight[..., None], 1e-6)
```

Listing 4: Distance-transform feather blend.

Quality metrics reported for every pair: initial matches, RANSAC inliers, inlier ratio, mean reprojection error, and overlap SSIM.

4. Interactive application

Six tabs: How it works, Build a panorama, Compare detectors, Robustness lab, Saved results, and Project report. The last of those embeds the full CVPR paper; the screenshots below are the walkthrough a marker can click through on the live URL.

How it works | Build a panorama | Compare detectors | Robustness lab | Saved results | Project report

How a panorama is built

Classical computer vision — not a pretrained recogniser

Phones make panoramas look like magic. Under the hood they are doing a very ordinary thing: **find the same landmarks in two photos, throw away the bad guesses, then stretch one photo until those landmarks line up.**

This app walks through that recipe with OpenCV. Nothing here is YOLO, nothing here is a black-box `Stitcher` class. Each step is a classical technique.

1. Landmarks

SIFT and ORB look for corners and blobs that stay recognisable when you move the camera. Think of unique bricks, window corners, signs.

2. Honest matches

Many landmark pairs are wrong. RANSAC keeps only the matches that agree on one geometric stretch — the homography.

3. One canvas

Each photo is warped onto the middle photo's coordinate system and the overlap is blended so the join is hard to see.

Overlapping photos → Prepare → Detect and describe → Match → RANSAC → Homography → Warp to the middle photo → Feather blend → Panorama

What you can do here

- **Build a panorama** — drop in three or more overlapping photos and step through keypoints, raw matches, RANSAC inliers, the warp, and the final stitch.
- **Compare detectors** — SIFT versus ORB on the same pair: keypoints, matches, inliers, inlier ratio, time, panorama quality.
- **Robustness lab** — rotate, scale, or darken one photo and watch the inlier ratio fall. Viewpoint uses a separate photo set.
- **Saved results** — tables and figures written by `scripts/run_experiments.py`.
- **Project report** — the full technical report as a PDF.

Live app: feature-based-panorama.streamlit.app · source: github.com/qbentil/feature-based-panorama

ⓘ The bundled scenes are real University of Ghana campus photographs from Wikimedia Commons (Balme Library, Dance Department, Night Market, Commonwealth Hall). They were not all shot as a dedicated panorama, so some sets overlap much more than others. Drop your own overlapping photos in the same folders when you can — the method does not change.

Figure 1: How it works: pipeline explanation (detect → match → RANSAC → warp → blend).

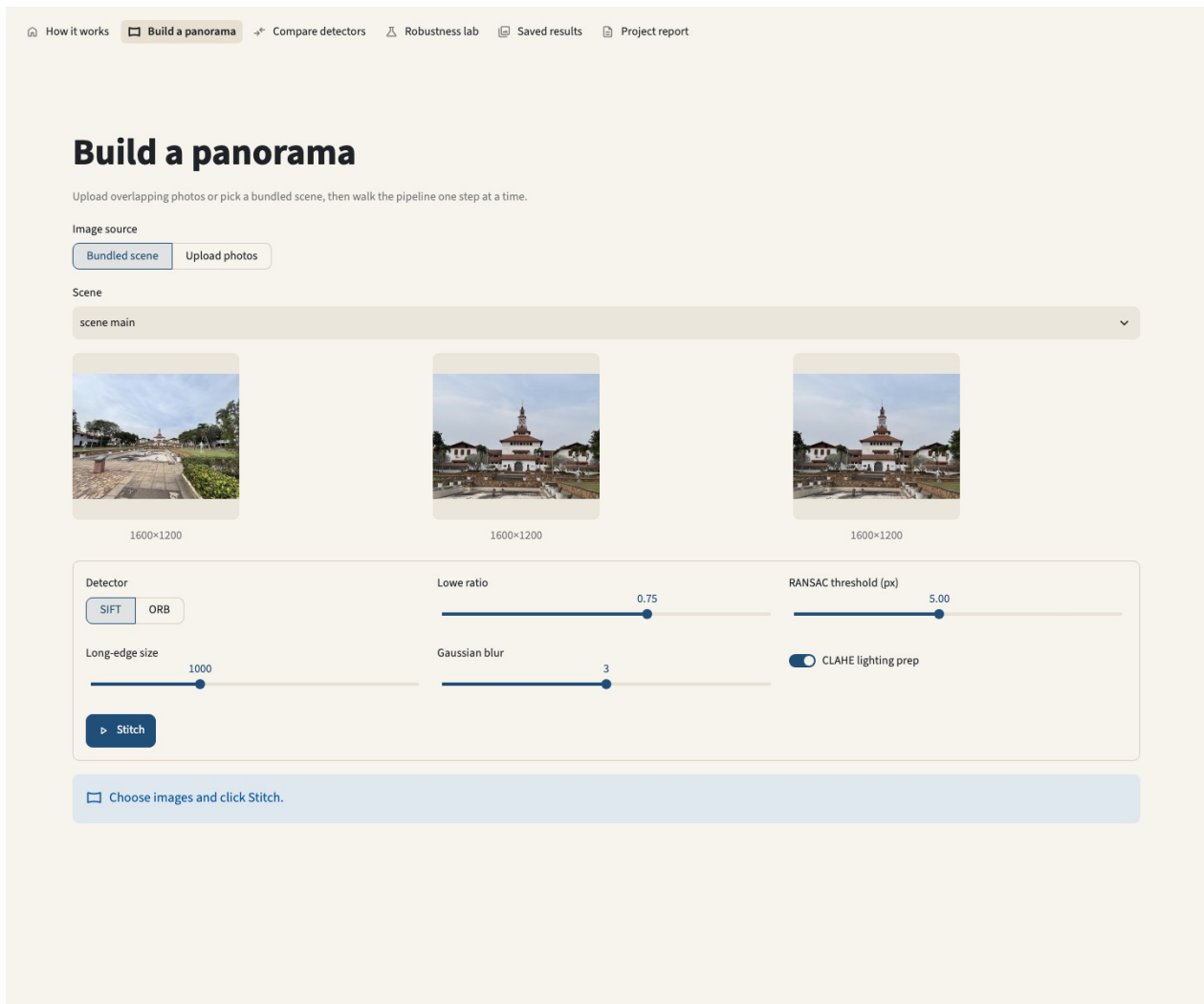


Figure 2: Build a panorama: bundled Balme Library views, SIFT, Lowe ratio 0.75, RANSAC 5 px.

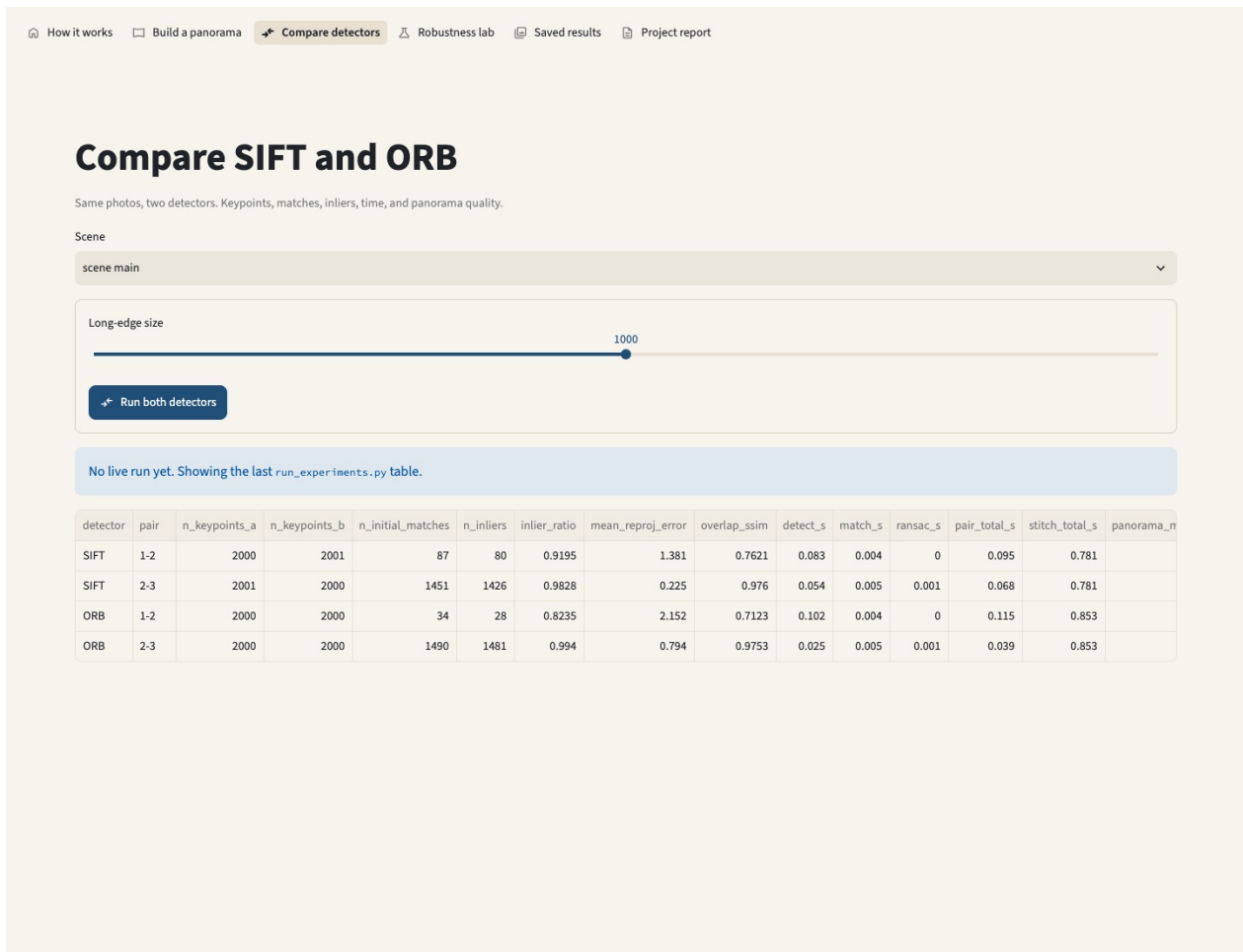


Figure 3: Compare detectors: side-by-side SIFT vs ORB table on the same pair.

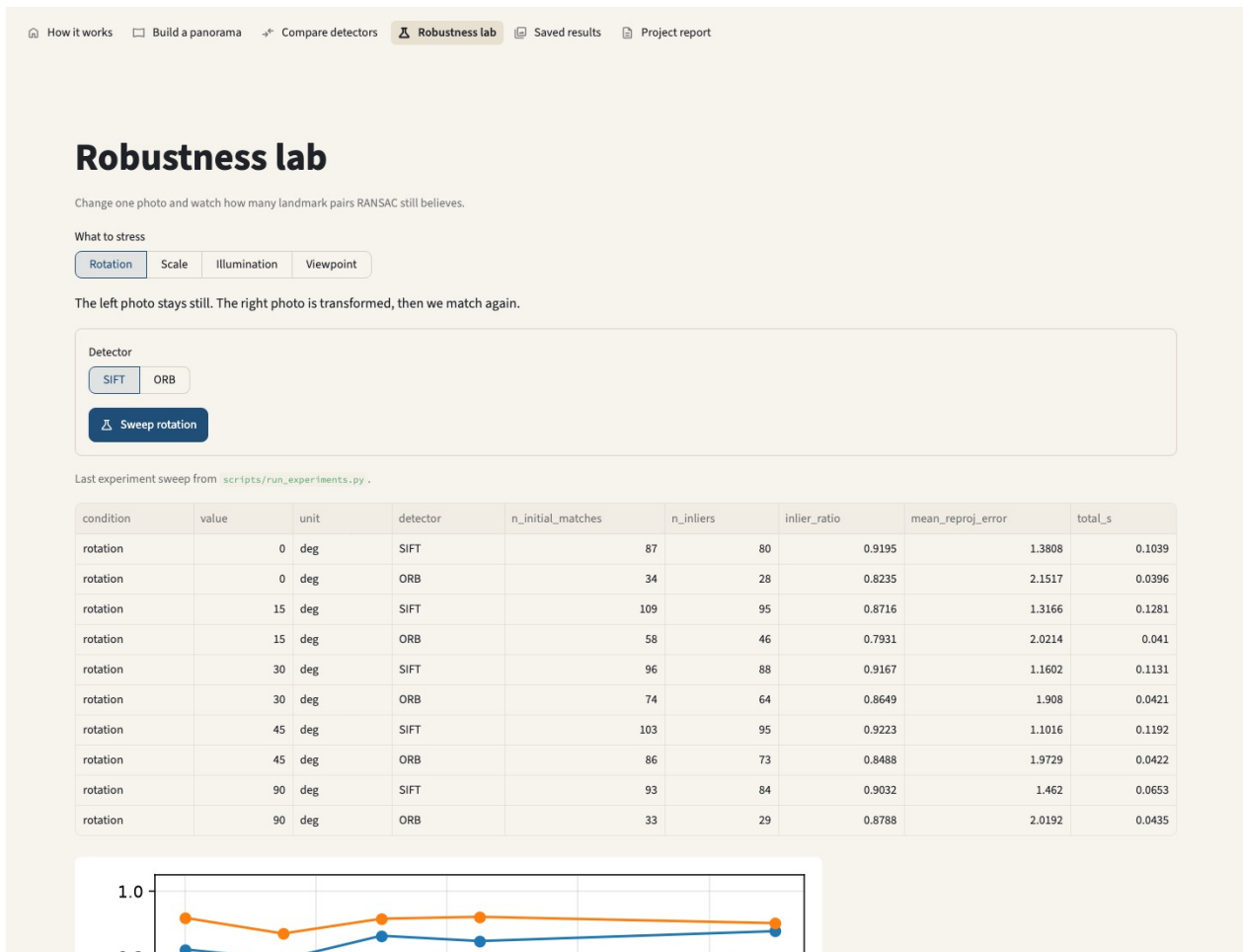


Figure 4: Robustness lab: rotation, scale, and illumination sweeps on a chosen pair.

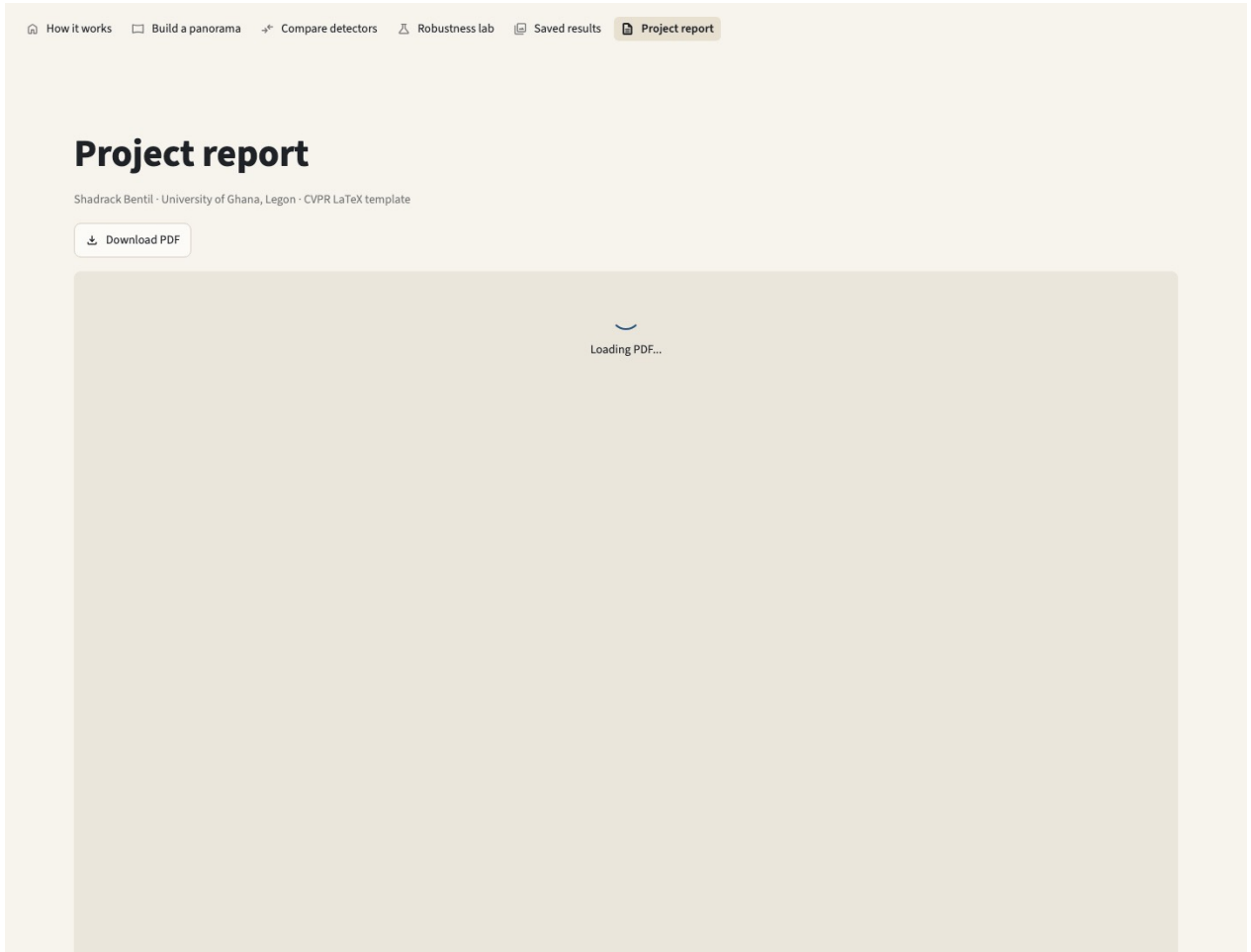


Figure 5: Project report tab: download and embed of the separate CVPR PDF (feature_based_panorama.pdf), not this packet.

5. Results

Primary scene: three Balme Library photographs (Wikimedia Commons, CC BY 4.0). Pair 2–3 is a near-duplicate facade; pair 1–2 is courtyard-to-facade with less overlap. Mean inlier ratio across the two consecutive pairs: SIFT 0.951, ORB 0.909. Full stitch about 0.78–0.85 s. Brightness gain 0.35 leaves SIFT with zero inliers. Dance Department viewpoint frames barely overlap.



Figure 6: Input views for the primary stitch (Balme Library, left to right).

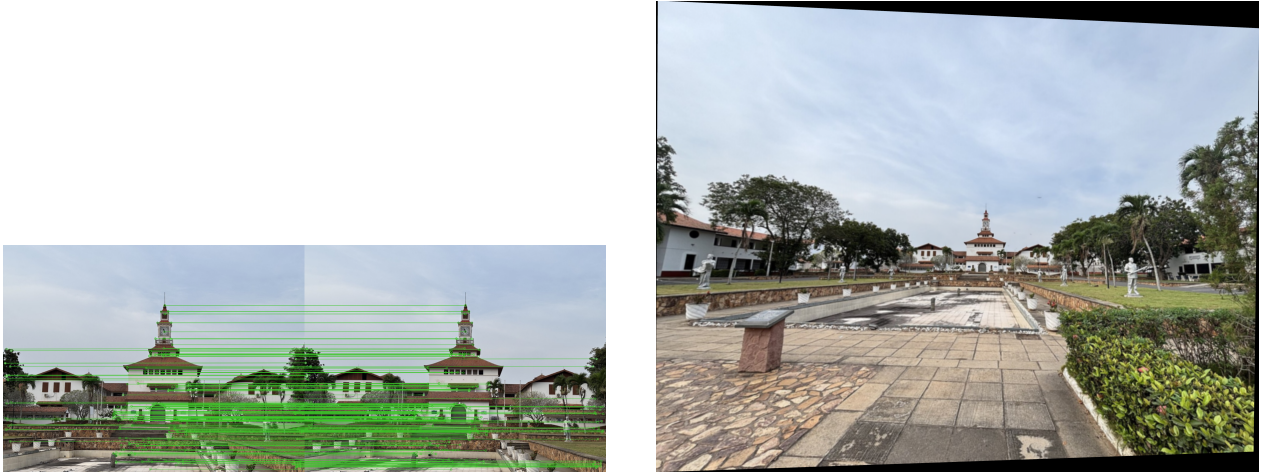


Figure 7: Left: SIFT inliers after RANSAC on a consecutive pair. Right: stitched panorama in the middle-image frame.

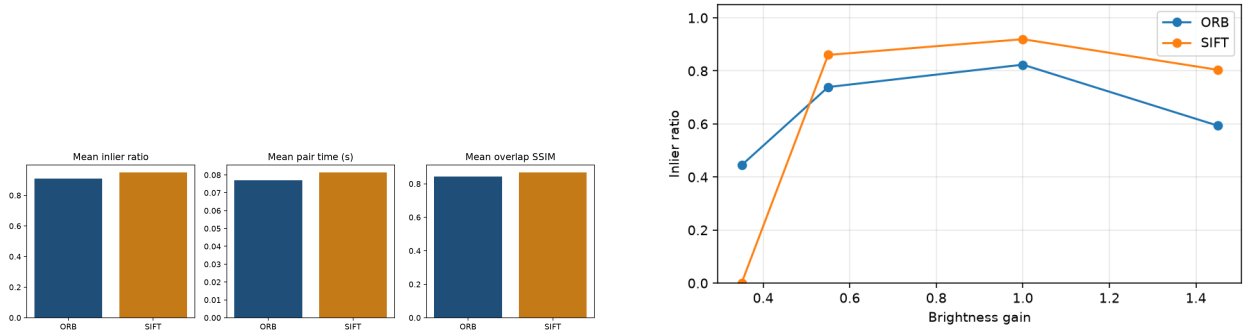


Figure 8: Left: SIFT vs ORB summary on Balme Library. Right: illumination sweep (inlier ratio vs brightness gain).

Table 1: Detector comparison on Balme Library consecutive pairs.

Detector	Pair	Initial	Inliers	Ratio	SSIM
SIFT	1–2	87	80	0.920	0.762
SIFT	2–3	1451	1426	0.983	0.976
ORB	1–2	34	28	0.824	0.712
ORB	2–3	1490	1481	0.994	0.975

SIFT is the tighter geometric estimator on the harder pair (reprojection 1.38 px vs 2.15 px for ORB on pair 1–2). ORB remains competitive when the facade is almost duplicated (pair 2–3). Both methods need genuine overlap: the Dance Department viewpoint set is a negative control and does not produce a usable panorama.

6. How to reproduce

```

pip install -r requirements.txt
python scripts/run_experiments.py
streamlit run streamlit_app.py

```

Or open the live URL in Section 1. Bundled photographs can be replaced with phone captures (about 30–50% overlap, captured left to right) in `data/scenes/`. Experiment CSVs land in `report/results/`; figures land in `report/figures/`. The CVPR paper is rebuilt with python scripts/`build_report.py`; that command also rebuilds this packet from `report/submission.tex`.